# Assignment 4 – SDfR

Group SDfR-PPD 16
Srikrishnan Srinivasan (s3603288); Raghavendran Srinivasan (s3656721)

March 2026

## Package delivery

This program implements the cost calculation for package delivery services. There are two types of package services: Two day package and Overnight Package. The cost of delivery depends for each of the services as follows:

$$\text{Two day} = 2.5(\text{Weight_kg}) + 5$$
$$\text{Overnight} = \text{Two_day} + 1.1(\text{Weight_kg})^2$$

To implement this we have implemented object oriented programming by creating Classes `Package, TwoDayPackage, OvernightPackage`. The program contains the functions: `CalculateCost()` `and printCost()`

### File Structure and Build Instructions

- `include/Package.h` : Contains class definitions and function prototype
- `src/Package.cpp` : Defines the functions
- `src/main.cpp` : File to call the functions

You can build the program using:

```
g ++ src /*. cpp - iquote include -o o.exe
```

and executed in the terminal by running

```
./o.exe
```

## Class Hierarchy

As mentioned above the program uses an object-oriented design using three classes: `Package, TwoDayPackage, OvernightPackage`. The base class is `Package`, which stores common in-

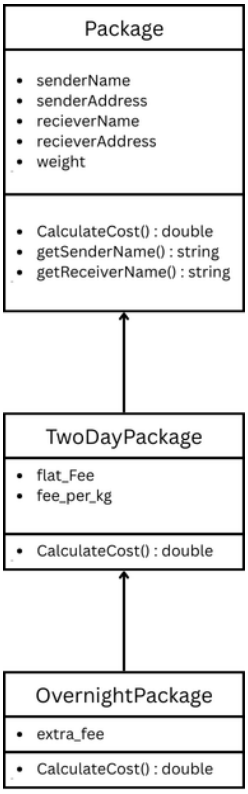


Figure 1: UML Diagram of Class Hirerachy (arrow points towards parent class)

formation about a package, while the derived classes implement the specific cost calculation methods. The relationship between the classes is shown below in Figure 1.

### Package Class

This class represents a generic package and stores common attributes shared by all packages. All attributes are listed as `protected` to allow these attributes to be accessed by derived classes, while still preventing direct access from outside the class hierarchy. The attributes include:

- Sender Name
- Sender Address
- Reciever Name
- Reciever Address
- weight

The Methods include:

- calculateCost() : Declared as `virtual` function. It allows allows derived classes to provide their own implementation of the function. This allows different types of package to have their own cost calculation. This is called **runtime polymorphism**. Additionally, the

`override` is used to indicate the function is intended to override a virtual function defined in the base class.

- getSenderName() : returns the sender's name
- getReceiverName() : returns the receiver's name

### TwoDayPackage Class

Inherits from `Package Class`, and adds `private` additional parameters `(flatFee & feeperkg)` for cost calculation.

### OvernightPackage Class

Inherits from `TwoDayPackage Class` because the overnight delivery cost is based on the two-day delivery cost with an additional charge.

## Design Decisions

The use of object-oriented programming provides several advantages in this program.

- **Encapsulation:** Each class encapsulates the data and behavior related to package shipping.
- **Inheritance:** The derived classes reuse functionality from the base class.
- **Polymorphism:** Allows different package types to be handled using a single interface (Package*)

## Test Cases and Results

### Regular Test Cases

The following regular test cases were tested and they results match the expected costs as shown in Figure 2:

Table 1: Test packages used in the program

| **Package Type** | **Sender** | **Sender Address** | **Receiver** | **Receiver Address** | **Weight (kg)** | **Expected Cost (€)** |
|---|---|---|---|---|---|---|
| TwoDayPackage | Alice | 123 Main St | Bob | 456 Elm St | 10 | 30.0 |
| TwoDayPackage | Charlie | 789 Oak St | Dave | 321 Pine St | 5 | 17.5 |
| OvernightPackage | Eve | 654 Maple St | Frank | 987 Cedar St | 3 | 22.4 |

```
Package from Alice to Bob costs: 30
Package from Charlie to Dave costs: 17.5
Package from Eve to Frank costs: 22.4
```

Figure 2: Results of regular test cases

### Edge Cases

We tested edge cases for very small weight and very large weight. Results can be seen in Figure 3.

Table 2: Edge Case Test Packages

| Package Type | Sender | Sender Address | Receiver | Receiver Address | Weight (kg) | Expected Cost (€) |
|---|---|---|---|---|---|---|
| TwoDayPackage | SmallSender | 1 Start St | SmallReceiver | 1 End St | 0.1 | 5.25 |
| OvernightPackage | SmallSender | 1 Start St | SmallReceiver | 1 End St | 0.1 | 5.261 |
| TwoDayPackage | LargeSender | 100 Start St | LargeReceiver | 100 End St | 100 | 255.0 |
| OvernightPackage | LargeSender | 100 Start St | LargeReceiver | 100 End St | 100 | 11255.0 |

```
Package from SmallSender to SmallReceiver costs: 5.25
Package from SmallSender to SmallReceiver costs: 5.261
Package from LargeSender to LargeReceiver costs: 255
Package from LargeSender to LargeReceiver costs: 11255
```

Figure 3: Results of edge cases

### Invalid Cases

The following Invalid cases were covered and tested against at, `main.cpp` contains these test cases and the results are show in Figure 4.

- **Invalid weights:** Packages with negative and zero weight were considered invalid. The program brings up the delivery details while re-entering the weight.
- **Empty strings:** The program catches these empty strings and sequentially fills them up, while relating to the delivery details. In-depth correction of this will be feasible with a unique delivery ID.
- **Identical Addresses:** Can't send a package to origin, and we ignored same name possibility as it might be possible. Cites package delivery details while re-entering the values.

Table 3: Invalid Test Packages

| Package Type | Sender | Sender Address | Receiver | Receiver Address | Weight (kg) |
|---|---|---|---|---|---|
| TwoDayPackage | Charlie | 789 Oak St | Dave | 321 Pine St | -5 |
| OvernightPackage | | 111 Blank Ave | | 222 Nowhere Rd | 2 |
| TwoDayPackage | Gary | Same House St | Gina | Same House St | 4 |

```
❯ ./a.out
Package from Alice to Bob costs: 30
Package from Charlie to Dave costs: 17.5
Package from Eve to Frank costs: 22.4

Edge test cases
Package from Alice to Bob costs: 30
Package from Charlie to Dave costs: 17.5
Package from Eve to Frank costs: 22.4
Package from SmallSender to SmallReceiver costs: 5.25
Package from SmallSender to SmallReceiver costs: 5.261
Package from LargeSender to LargeReceiver costs: 255
Package from LargeSender to LargeReceiver costs: 11255

 Invalid cases
Warning: Invalid weight entered!
Please enter a valid weight for the package Charlie to Dave(>0) in Kilograms: 0
Warning: Invalid weight entered!
Please enter a valid weight for the package Charlie to Dave(>0) in Kilograms: -7
Warning: Invalid weight entered!
Please enter a valid weight for the package Charlie to Dave(>0) in Kilograms: 8

 Invalid case

Warning: Sender name is empty!
Please enter the sender's name for this new package: John

Warning: Receiver name is empty for the package from John!
Please enter the receiver's name: David

 Invalid case
Warning: Sender and Receiver Addresses for the delivery Gary to Gina the same!
Please re-enter the receiver's address: Same House St
Warning: Sender and Receiver Addresses for the delivery Gary to Gina the same!
Please re-enter the receiver's address: Same House St
Warning: Sender and Receiver Addresses for the delivery Gary to Gina the same!
Please re-enter the receiver's address: New House St
Package from Alice to Bob costs: 30
Package from Charlie to Dave costs: 17.5
Package from Eve to Frank costs: 22.4
Package from SmallSender to SmallReceiver costs: 5.25
Package from SmallSender to SmallReceiver costs: 5.261
Package from LargeSender to LargeReceiver costs: 255
Package from LargeSender to LargeReceiver costs: 11255
Package from Charlie to Dave costs: 25
Package from John to David costs: 14.4
Package from Gary to Gina costs: 15
```

Figure 4: Results of Edge test cases

## 4.3 Questions

### (a) Inheritance

Inheritance is used to reuse code. We have `TwoDayPackage` and `OvernightPackage` which are just types of `Package`s. They share the same core attributes such as `senderName, senderAddress, receiverName, receiverAddress` and `weight`. The need to write code for all this can be done once and forever with the help of inheritance. Derived classes inherit these attributes without any extra effort. It also allows for polymorphism where base classes and derived classes can be accessed through the same regime.

### (b) Protected access

- `Public` access specifiers are open to everyone, any code can access and modify the members from anywhere.
- `Private` access specifiers are locked down until a function is strictly specified to use them. Derived classes cannot access them either.
- `Protected` access specifiers are a mix of `Private` (to code outside) and `Public` (with derived classes) access specifiers.

### (c)Composition and Inheritance

Composition explains how an class is composed of other objects. In case of `computer` it is made of multiple components, hence its composition can be described with parts like `CPU, RAM, GPU`. It is seen that composition describes a 'has a' type relationship.

Inheritance is when one classes derive from another class. In case of `computer`, we could have classes like `Laptop and Desktop`, since they are also computers and attributes can be derived from the base class. The attributes like `CPU, RAM, GPU` are inherited into the child classes. It is seen that inheritance describes a 'is a' type relationship.

### (d) Insert Calls to Destructor

For a local object when an object is normally created inside a function, the compiler inserts the destructor call right after the object goes out of scope. Global objects are automatically destructed when the entire program finishes executing. For dynamic objects, the destructor is called when the delete command is run on the pointer.

### (e) Virtual Destructor

It is a destructor in a base class. It is necessary when there is class hierarchy and the deletion of a derived object through a pointer of the base class. Without the virtual destructor, only the static memory is deleted and might cause any memory or resources tied to the derived classes (in our case) to be left behind. Virtual tells the compiler to look at the dynamic link and remove the whole thing.

## AI Declaration

We declare the use of AI tools to help understand the use of virtual functions which can be overidden.